Practical – 7

Aim-1: Form validator

Define @NotBlank and @MaxLength(int) for fields and annotate a Signup Form. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.

Code:

Main.java

```java
```java
import java.lang.annotation.*;
import java.lang.reflect.Field;

// Custom annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

// Class containing annotated fields
class Student {
    @Column(name = "id")
    private int id;

    @Column(name = "name")
    private String name;

    @Column(name = "course")
    private String course;

    @Column(name = "age")
    private int age;

    public void display() {
        System.out.println("ID     : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Course : " + course);
        System.out.println("Age    : " + age);
```
```

Main.java

```
class Student {
    public void display() {
        System.out.println("Course : " + course);
        System.out.println("Age    : " + age);
    }
}

public class ColumnMappingDemo {

    public static void main(String[] args) {

        // Header row
        String[] headers = {"id", "name", "course", "age"};

        // Data row
        String[] data = {"101", "Kavya", "IT", "19"};

        Student student = new Student();

        try {
            Field[] fields = Student.class.getDeclaredFields();

            for (Field field : fields) {

                // Check whether field has @Column
                if (field.isAnnotationPresent(Column.class)) {

                    Column column = field.getAnnotation(Column.class);
                    String columnName = column.name();
```

Main.java

```
public class ColumnMappingDemo {
    public static void main(String[] args) {
        try {
            for (Field field : fields) {
                if (field.isAnnotationPresent(Column.class)) {

                    int index = -1;

                    // Find matching column
                    for (int i = 0; i < headers.length; i++) {
                        if (headers[i].equals(columnName)) {
                            index = i;
                            break;
                        }
                    }

                    // Missing column handling
                    if (index == -1) {
                        System.out.println(
                            "Warning: Column '" + columnName +
                            "' is missing."
                        );
                        continue;
                    }

                    // Allow access to private field
                    field.setAccessible(true);

                    // Convert data according to field type
                    if (field.getType() == int.class) {
```

```
public class ColumnMappingDemo {
    public static void main(String[] args) {
        try {
            for (Field field : fields) {
                if (field.isAnnotationPresent(Column.class)) {
                    );
                    continue;
                }

                // Allow access to private field
                field.setAccessible(true);

                // Convert data according to field type
                if (field.getType() == int.class) {
                    field.setInt(student, Integer.parseInt(data[index]));
                } else if (field.getType() == String.class) {
                    field.set(student, data[index]);
                }
            }
        }

        System.out.println("Student Details:");
        student.display();

    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}
```

Output:

```
Student Details:
ID     : 101
Name   : Kavya
Course : IT
Age    : 19
```

Learning Outcomes – Problem 1: Form Validator

After completing this task, students will be able to:

1. Understand and create **custom annotations** in Java.
2. Use `@NotBlank` and `@MaxLength` to define **field validation rules**.
3. Use **reflection** to inspect fields and their annotations at runtime.
4. Validate an object dynamically without writing separate validation code for each field.
5. Generate and display a **list of validation errors** for invalid input.
6. Understand the basic concept behind annotation-based validation used in frameworks.

Aim-2 : Mini test runner

Define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.

Code :

Main.java

```java
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}

class MyTests {

    @Run
    public void testOne() {
        System.out.println("Test One executed");
    }

    @Run
    public void testTwo() {
        System.out.println("Test Two executed");
    }

    public void notATest() {
        System.out.println("This method should not run");
    }

    @Run
    public void testThree() {
        System.out.println("Test Three executed");
```

Main.java

```java
class MyTests {
}

public class Main {

    public static void main(String[] args) {

        int count = 0;

        try {
            MyTests testObject = new MyTests();

            Method[] methods = MyTests.class.getDeclaredMethods();

            for (Method method : methods) {

                if (method.isAnnotationPresent(Run.class)
                        && method.getParameterCount() == 0) {

                    method.invoke(testObject);
                    count++;
                }
            }

            System.out.println("Total @Run methods executed: " + count);

        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
```

Output :

```
Test One executed
Test Two executed
Test Three executed
Total @Run methods executed: 3
```

Learning Outcomes – Problem 2: Mini Test Runner

After completing this task, students will be able to:

1. Create and use **custom annotations** in Java.
2. Understand how **reflection** can inspect methods at runtime.
3. Identify methods marked with a specific annotation.
4. Invoke methods dynamically using `Method.invoke()`.
5. Count and report the number of executed test methods.
6. Understand the basic concept behind how **JUnit uses `@Test` annotations** to run test methods.

Aim – 3 : For Advanced Learners – home

Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.

Code :

Main.java

```java
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.annotation.ElementType;
import java.lang.reflect.Field;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class Student {

    @Column(name = "id")
    private int id;

    @Column(name = "name")
    private String name;

    @Column(name = "course")
    private String course;

    @Column(name = "age")
    private int age;

    public void display() {
        System.out.println("ID     : " + id);
        System.out.println("Name   : " + name);
        System.out.println("Course : " + course);
```

CEUE20325IT103



```
Main.java
13  class Student {
27      public void display() {
31          System.out.println("Age     : " + age);
32      }
33  }
34
35  public class Main {
36
37      public static void main(String[] args) {
38
39          String[] headers = {"id", "name", "course", "age"};
40          String[] data = {"101", "Kavya", "IT", "19"};
41
42          Student student = new Student();
43
44          try {
45              Field[] fields = Student.class.getDeclaredFields();
46
47              for (Field field : fields) {
48
49                  if (field.isAnnotationPresent(Column.class)) {
50
51                      Column column = field.getAnnotation(Column.class);
52                      String columnName = column.name();
53
54                      int index = -1;
55
56                      // Find matching header
57                      for (int i = 0; i < headers.length; i++) {
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        try {
            for (Field field : fields) {
                if (field.isAnnotationPresent(Column.class)) {
                                index = i;
                                break;
                            }
                        }

                        // Handle missing column
                        if (index == -1) {
                            System.out.println(
                                "Warning: Column '" + columnName + "' is missing."
                            );
                            continue;
                        }

                        field.setAccessible(true);

                        // Set value according to field type
                        if (field.getType() == int.class) {
                            field.setInt(
                                student,
                                Integer.parseInt(data[index])
                            );
                        } else if (field.getType() == String.class) {
                            field.set(student, data[index]);
                        }
                    }
```

```java
public class Main {
    public static void main(String[] args) {
        try {
            for (Field field : fields) {
                if (field.isAnnotationPresent(Column.class)) {
                    continue;
                }

                field.setAccessible(true);

                // Set value according to field type
                if (field.getType() == int.class) {
                    field.setInt(
                        student,
                        Integer.parseInt(data[index])
                    );
                } else if (field.getType() == String.class) {
                    field.set(student, data[index]);
                }
            }
        }

        System.out.println("Student Details:");
        student.display();

    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}
}
```

Output :

```
Student Details:
ID      : 101
Name    : Kavya
Course : IT
Age     : 19
```

Learning Outcomes – Problem 3

After completing this task, students will be able to:

1. Create and use a **custom `@Column` annotation** in Java.
2. Understand how **reflection** can access class fields at runtime.
3. Match object fields with **column names dynamically**.
4. Populate object fields using data from a **header row and data row**.
5. Handle **missing columns** without stopping the program.
6. Understand how annotations and reflection can be used for **data mapping** in real applications.